

Mini Assessment – GenAI Mastery

API Error Handling and Safe JSON Access

Name SAMPATHI YASWANTH

Reg. No. 99230041039

Department CSE AIML(4th Year)

Q1. If an API returns 429, what should your program do?

HTTP 429 (Too Many Requests) means the program has sent too many requests to the API within a certain period. The program should not immediately keep sending requests, because that can make the rate-limit problem worse.

The program should wait before retrying. If the API provides a Retry-After header, the program should follow that value. Otherwise, it can use exponential backoff, such as waiting 1 second, then 2 seconds, 4 seconds, and so on. It should also limit the number of retries.

Flow:

```
API returns 429
|
Wait for the required time
|
Retry the request
|
If it fails again -> increase the waiting time
```

In simple words: A 429 response means “slow down.” The program should wait and retry rather than continuously sending requests.

Q2. If response.json() fails, what should your program do?

response.json() is used to convert an API response into JSON data. However, sometimes the server may return invalid JSON, empty content, HTML, or another unexpected format. In that situation, response.json() can raise a parsing error.

The program should catch the error and handle it gracefully instead of crashing. It can log the error and inspect response.text to understand what the server actually returned. The program can then return or display an appropriate error message.

Example:

```
try:
    data = response.json()
except ValueError:
    print("Invalid JSON response")
    print(response.text)
```

In simple words: If the response is not valid JSON, the program should handle the error safely instead of assuming the response is always valid.

Q3. If a JSON key is missing, why might `.get()` be safer than `[]`?

When accessing a key from a JSON object in Python, `[]` and `.get()` behave differently if the key does not exist.

Using `[]` directly:

```
name = data["name"]
```

If "name" is missing, Python raises a `KeyError`, which can cause the program to stop if the error is not handled.

Using `.get()`:

```
name = data.get("name")

# You can also provide a default value:
name = data.get("name", "Unknown")
```

If "name" is missing, `.get()` returns `None` instead of raising an error.

In simple words: `.get()` is safer because it handles missing keys without immediately raising an error, making the program more robust when working with API responses.

Quick Summary

Question	Correct Approach
API returns 429	Wait and retry using Retry-After or exponential backoff
<code>response.json()</code> fails	Catch the parsing error and handle the invalid response
JSON key is missing	Use <code>.get()</code> to safely handle missing keys